

UNIT - II

①

Parsers

- The second phase of the compiler is assign the task of checking the syntax and thus it is called as syntax analyzer or parser.
- Syntax analyzer is a program that takes tokens from the lexical analyzer and checks the syntax of the statements. If the statements are syntactically correct, the phase generates a syntax tree but if there are errors, then the errors are generated and reported.

Role of Parser

- Parser or Syntax analyzer is the program which performs syntax analysis or parsing.
- Parsing is an important phase of compiler-design, which obtains string of tokens produced as output of Lexical Analysis.

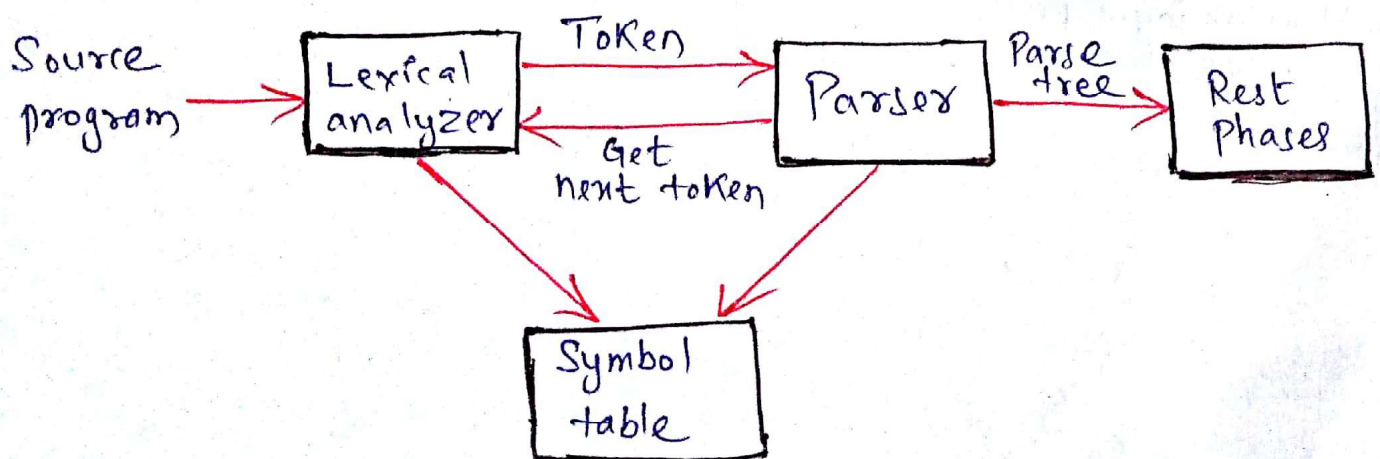


fig. Position of parser in compiler model.

②

• There are different types of parsers based on the way they parse the input.

(1) Cocke - Younger - Kasami algorithm

(2) Earley's algorithm

(3) Top-down or Bottom-up parser.

First two methods are inefficient in production of compiler, hence commonly top-down and bottom-up parser used.

Top-down parser build parse trees from the top (root) to the bottom (leaves).

Bottom-up parser starts from the leaves and work their way up to the root.

Ambiguous Grammar

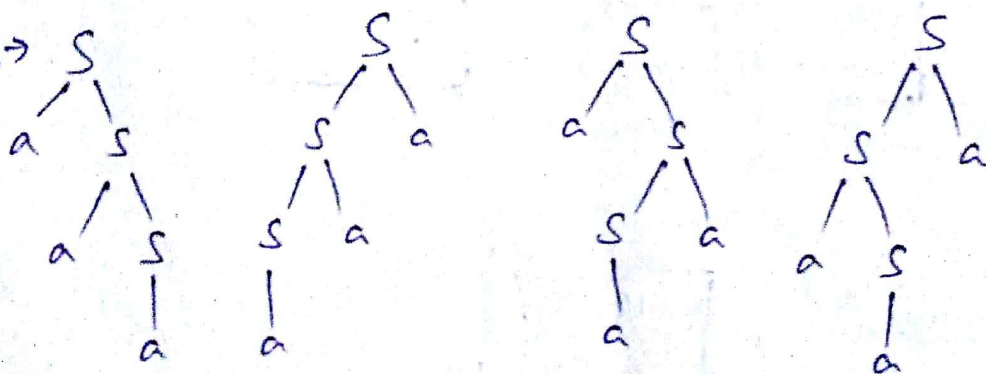
Consider the following grammar

$$S \rightarrow aS / Sa / a$$

String $w = aaa$

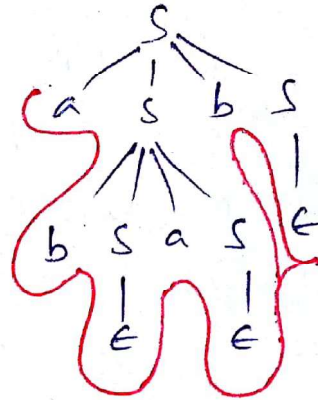
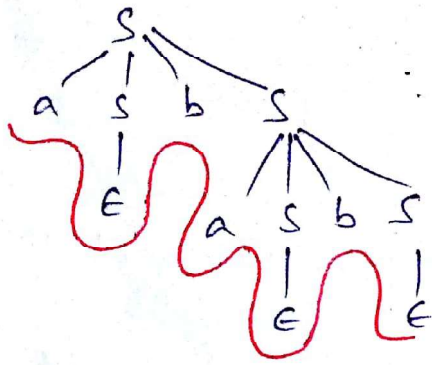
How many parse tree's possible ?

Solution $\Rightarrow$



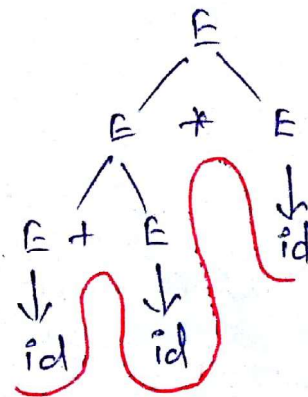
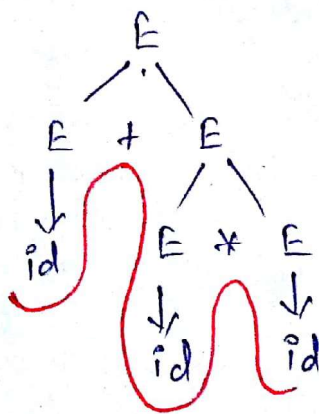
Given grammar is ambiguous grammar, because to generate $w = aaa$ more than one parse tree is available.

Example (2): $S \rightarrow aSbS / bSaS / \epsilon$
 $w = abab$



$\therefore$ Ambiguous grammar

Example (3): $E \rightarrow E + E / E * E / id$
 $w = id + id * id$



$\therefore$ Ambiguous grammar

If in a grammar, there exists atleast one string which give more than one parse tree, then the given grammar is ambiguous.

(4)

Leftmost Derivation And Rightmost Derivation

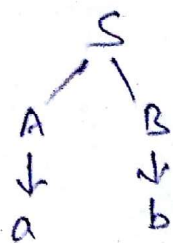
- While deriving the string, if we always substitute leftmost variable, then it is called Leftmost derivation.
- While deriving the string, if we always substitute rightmost variable, then it is called rightmost derivation.

$$S \rightarrow AB$$

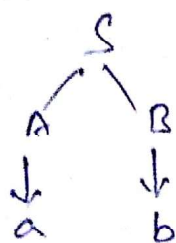
$$A \rightarrow a$$

$$B \rightarrow b$$

$$w = ab$$



LMD tree



RMD tree

LMD	RMD
S	S
AB	AB
aB	Ab
ab	ab

Note: If the given grammar is unambiguous grammar, then leftmost derivation tree is equal to rightmost derivation tree.

If the given grammar is ambiguous grammar, then leftmost derivation tree need not be equal to rightmost derivation tree.

Conversion from Ambiguous to Unambiguous Grammar

$$\begin{aligned} E &\rightarrow E + E / E * E / id \\ w &= id + id * id \end{aligned} \quad \left. \vphantom{\begin{aligned} E &\rightarrow E + E / E * E / id \\ w &= id + id * id \end{aligned}} \right\} \text{Ambiguous Grammar}$$

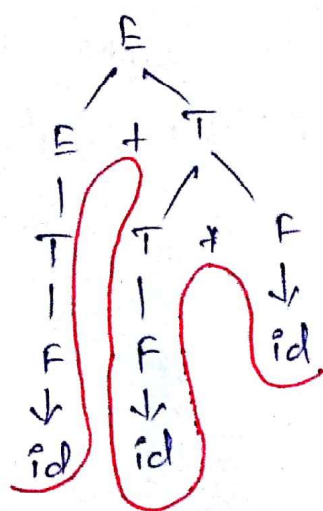
⇓ According to C

$$\begin{aligned} E &\rightarrow E + T / T \\ T &\rightarrow T * F / F \\ F &\rightarrow id \end{aligned} \quad \left. \vphantom{\begin{aligned} E &\rightarrow E + T / T \\ T &\rightarrow T * F / F \\ F &\rightarrow id \end{aligned}} \right\} \text{Unambiguous Grammar}$$

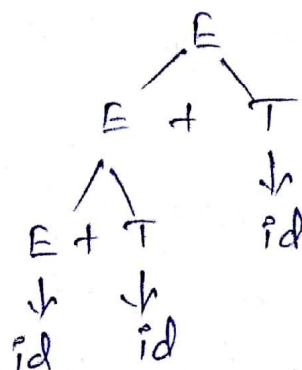
Now '*' has higher priority than '+'

$$w = id + id * id$$

∴



$$\text{If } w = id + id + id$$



∴ '+' is having left-to-right associativity.

$$\begin{aligned} \therefore E &\rightarrow E + T / E - T / T \\ T &\rightarrow T * F / T / F / F \\ F &\rightarrow G \uparrow F / G \\ G &\rightarrow id. \end{aligned}$$

Note: (1) '+' and '-' both have same priority but associativity is left to right.

- 6) (2) '*' and '/' both have same priority but associativity is Left to Right.
- (3) '↑' has higher priority and associativity is Right-to Left.

Q. Convert the following Ambiguous Grammar to equivalent unambiguous Grammar.

$$E \rightarrow E + E \mid E * E \mid E / E \mid E - E \mid E \uparrow E \mid id$$

With the following Rules :

- $\uparrow$: Lower priority & Associativity is Left-Right.
- $*, +$: next-lower priority & Associativity is Right-Left
- $/$: next-lower priority & Associativity is Right-Left
- $-$: Higher priority & Associativity is Left-Right.

Solution :>

$$E \rightarrow E \uparrow T \mid T$$

$$T \rightarrow F * T \mid F + T \mid F$$

$$F \rightarrow G / F \mid G$$

$$G \rightarrow G - H \mid H$$

$$H \rightarrow id$$

Q. $R \rightarrow R + R \mid R . R \mid R^* \mid a \mid b \mid \epsilon$

According to $\subset$ language,

Union : lower priority
 concatenation : Middle priority
 Kleen closure : Higher priority

}

Associativity is left-Right

⑧ Elimination of Left Recursion

Example (1)

$$\begin{aligned} E &\rightarrow E + T / T \\ T &\rightarrow T * F / F \\ f &\rightarrow id \end{aligned}$$

Solution: $\Rightarrow$

$$\begin{aligned} E &\rightarrow E + T / T \\ &\Downarrow \\ E &\rightarrow TE' \\ E' &\rightarrow +TE' / \epsilon \end{aligned}$$

$$\begin{aligned} T &\rightarrow T * F / F \\ &\Downarrow \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' / \epsilon \end{aligned}$$

$$\therefore \begin{aligned} E &\rightarrow E + T / T \\ T &\rightarrow T * F / F \\ f &\rightarrow id \end{aligned} \Rightarrow$$

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' / \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' / \epsilon \\ f &\rightarrow id \end{aligned}$$

NOTE: $A \rightarrow A\alpha / B$

Solution:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \epsilon$$

Example (2)

$$\begin{aligned} S &\rightarrow (L) / a \\ L &\rightarrow L, S / S \end{aligned}$$

Solution:

$$\begin{aligned} L &\rightarrow L, S / S \\ &\Downarrow \\ L &\rightarrow SL' \\ L' &\rightarrow , SL' / \epsilon \end{aligned}$$

$$\begin{array}{l} \therefore S \rightarrow (L)/a \\ L \rightarrow L, S/S \end{array} \Rightarrow \begin{array}{l} S \rightarrow (L)/a \\ L \rightarrow SL' \\ L' \rightarrow , SL'/\epsilon \end{array}$$

Example 3:

$$\begin{array}{l} S \rightarrow aBbh \\ \underline{b} \rightarrow Bb/h \\ D \rightarrow EF \\ E \rightarrow g/\epsilon \\ F \rightarrow f/\epsilon \end{array}$$

Solution $\Rightarrow$

$$\begin{array}{l} S \rightarrow aBbh \\ B \rightarrow hB' \\ B' \rightarrow bB'/\epsilon \\ D \rightarrow EF \\ E \rightarrow g/\epsilon \\ F \rightarrow f/\epsilon \end{array}$$

Example 4:

$$\begin{array}{l} S \rightarrow A \\ \underline{A} \rightarrow \underline{A}d/\underline{A}e/\underline{A}f/ab/ac \\ B \rightarrow bBc/f \end{array}$$

Solution $\Rightarrow$

$$\begin{array}{l} S \rightarrow A \\ A \rightarrow aBA'/acA' \\ A' \rightarrow dA'/eA'/fA'/\epsilon \\ B \rightarrow bBc/f \end{array}$$

Example 5:

$$\begin{array}{l} S \rightarrow Aa/b \\ A \rightarrow Ac/sd/\epsilon \end{array}$$

(10) Solution →

$$S \rightarrow Aa/b$$

$$A \rightarrow Ac/Sd/E$$



$$S \rightarrow Aa/b$$

$$A \rightarrow Ac/Aad/bd/E$$

∴ After removing left recursion, we have

$$S \rightarrow Aa/b$$

$$A \rightarrow bda'/A'$$

$$A' \rightarrow E/cA'/adaA'$$

Left Factoring

$$S \rightarrow ab/ac/ad$$

$$w = ad$$

From the above grammar, to generate string $w = ad$ all the three productions are fighting because the 1st character in the string is given by all the three production.

This problem is known as Left factoring.

Elimination of Left Factoring

$$S \rightarrow ab/ac/ad$$

$$w = ad$$



$$S \rightarrow as'$$

$$s' \rightarrow b/c/d$$

Example (2):

$$S \rightarrow iEtS / iEtSeS / b$$

$$E \rightarrow a$$



$$S \rightarrow b / iEtSS'$$

$$S' \rightarrow e / eS$$

$$E \rightarrow a$$

Example (3):

$$S \rightarrow a / ab / abc / abcd$$



$$S \rightarrow as'$$

$$S' \rightarrow e / bs''$$

$$S'' \rightarrow e / cs'''$$

$$S''' \rightarrow e / d$$

Top-down Parser $\Rightarrow$

There are two types of Top-down parser

(1) Recursive Descent Parser

(2) Non-Recursive Descent Parser or LL(1).

(1) Recursive Descent Parser

- A recursive-descent parsing program consists of a set of procedures one for each non-terminal.
- Execution begins with the procedure for the start symbol which halts and announces success if its procedure body scans the entire input string.

(12)

Algorithm

void S()

{

choose a production of S i.e. $S \rightarrow x_1 x_2 x_3 \dots x_n$

for ($i = 1$ to n)

{

if (x_i is variable)

call procedure $x_i()$;

else if (x_i equals the current input symbol a)

advance the input to the next symbol

else

error (means choose another production of S)

}

- Recursive descent parser uses leftmost derivation.
- If the grammar contain left recursion, Recursive descent parser will go to infinite loop.
- lot of time is wasted in backtracking.
- Data structure used is stack.

Example $\Rightarrow$

$A \rightarrow abc / aBd / aAD$

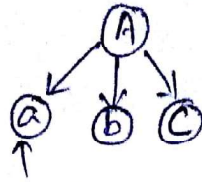
$B \rightarrow bB / \epsilon$

$C \rightarrow d / \epsilon$

$D \rightarrow a / b / \epsilon$

string $w = aaba$

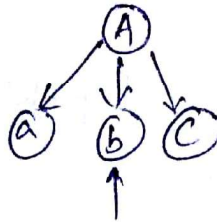
Input = aaba
↑



Here, input character matches the first character of the derived string.

Now increment the input pointer (reading next input character).

Input = aaba
 ↑



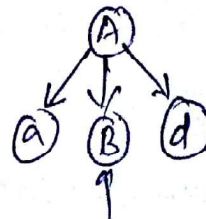
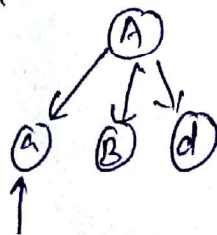
Here, input character is 'a' and the derived character is 'b'. Hence, we see that the production rule chosen is not the appropriate one.

So we need the backtracking.



Using next production rule $A \rightarrow aBd$ and start reading input again.

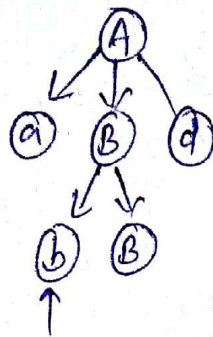
Input = aaba
 ↑



Now we encounter a non-terminal B in the tree.

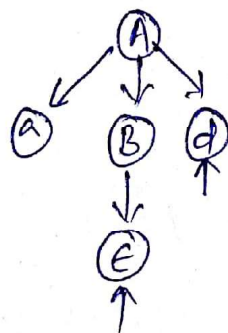
we use rule $B \rightarrow bB$.

14



This production gives 'b' as the next character which is not appropriate for the derivation.

Next production is $B \rightarrow \epsilon$.



Input = a a b a
 ↑

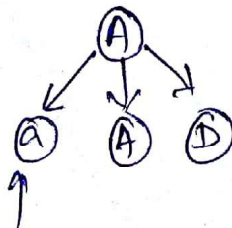
The current input character 'a' does not match the derived 'd'. So backtrack.

There are no productions remaining derived from B.

So backtrack to previous non-terminal i.e. A.

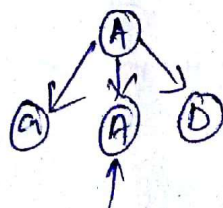
Now using $A \rightarrow aAD$ and start from beginning.

Input = a a b a
 ↑



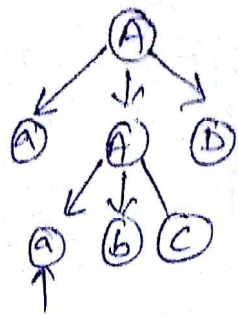
Increment input pointer

Input = a a b a
 ↑

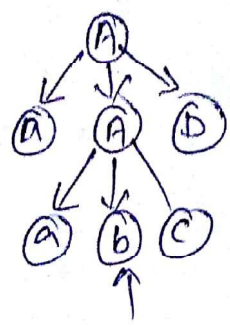


for non-terminal A using production rule $A \rightarrow abc$

Input = aaba
↑

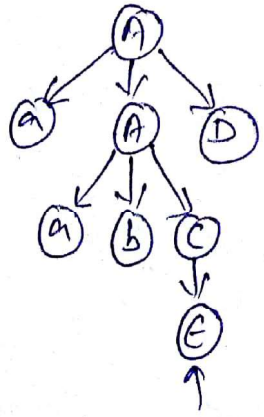


Input = aaba
↑

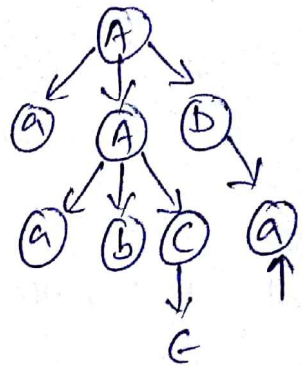


Input = aaba
↑

Substituting $C \rightarrow d$ will not be an appropriate step. So replace $C \rightarrow \epsilon$



Now $D \rightarrow a$



∴ The string aaba is accepted. and successful completion of parsing.

Question: Consider the grammar

$S \rightarrow cAd$
 $A \rightarrow ab|a$

String $w = cad$

Use Recursive Descent Parser to parse the given String.

Ans: Successful completion of string parsing

Q $E \rightarrow TE'$
 $E' \rightarrow +TE' / \epsilon$

Ans: Recursive Descent Parser Algorithm for above production is:

```

E()
{
    if (l == 'i')
    {
        match('i');
        E'();
    }
}

```

```

E'()
{
    if (l == '+')
    {
        match('+');
        match('i');
        E'();
    }
}

```

```

else
    return;
}

```

```

match(char t)
{
    if (l == t)
        l = getchar();
    else
        print("error");
}

```

```

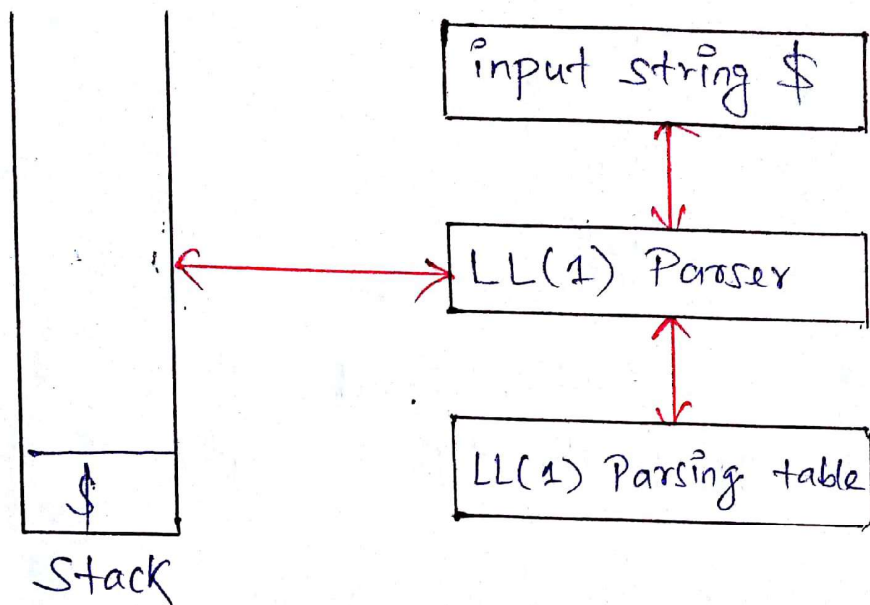
main()
{
    E();
    if (l == '$')
        printf("success");
}

```

Example: $i + i \$$

Non - Recursive Descent Parser (or) LL(1)

- A non-recursive descent parsing can be implemented by maintaining an input buffer, a stack and a parsing table rather than implicit implementation using recursive calls.



- LL(1) parsing table has a row for every non-terminal and a column for every terminal.
- The stack contains a sequence of grammar symbol with \$ at the bottom. Initially the stack contains the starting symbol of the grammar on the top.
- The first "L" in LL(1) stands for scanning the input from left to right.
The second "L" for producing a leftmost derivation.
The "1" for using one input symbol of lookahead at each step to make parsing action decisions.

LL(1) Parsing Algorithm

Let X is top-of-stack and a is lookahead symbol.

(i) If $(X == a == \$)$ then successful completion of parsing.

(ii) If $(X == a) \neq \$$ then pop TOS and increment input pointer.

(iii) If $(X \text{ is variable})$ then

see parsing table entry $M[X, a]$

If $M[X, a] = x \rightarrow uvw$

then replace x by uvw in reverse order.

(iv) If $M[X, a] = \text{blank}$, then Error.

Question (1): Apply LL(1) parsing algorithm on the following grammar to parse the given input string.

$$E \rightarrow TE'$$

$$E' \rightarrow \epsilon / + TE'$$

$$T \rightarrow FT'$$

$$T' \rightarrow \epsilon / * FT'$$

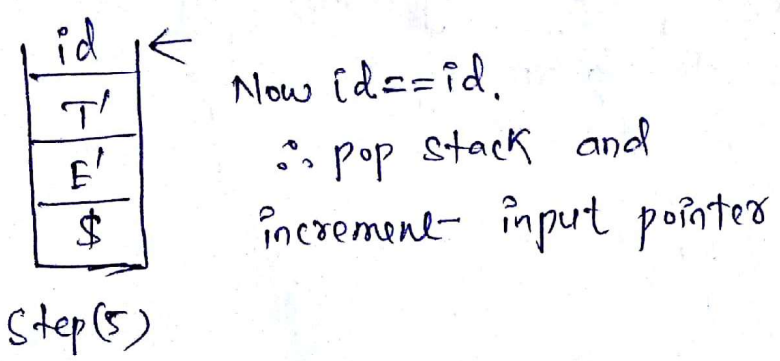
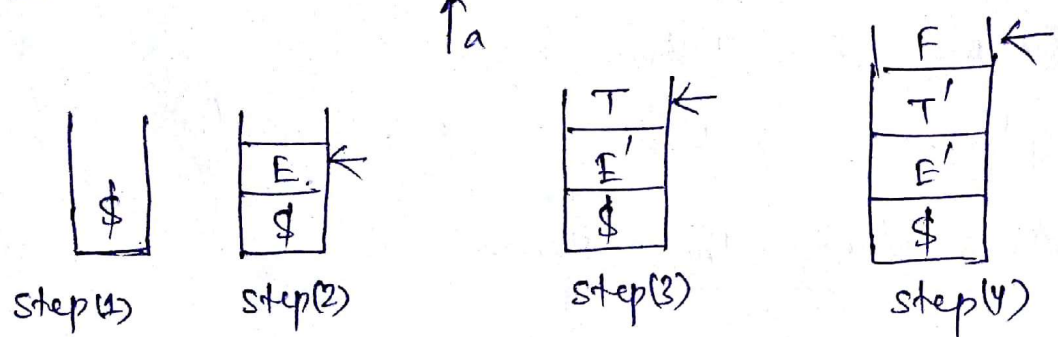
$$F \rightarrow id / (E)$$

string $w = id + id * id$

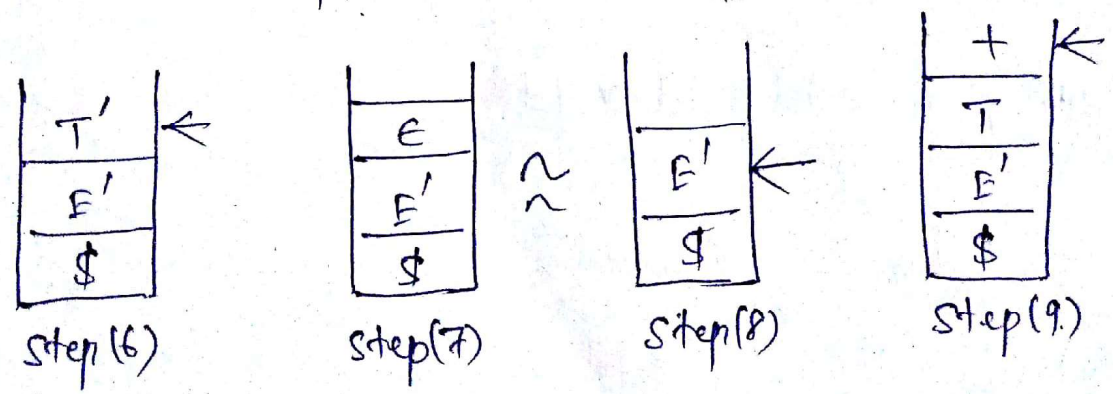
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow E$	$E' \rightarrow E$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow E$	$T' \rightarrow FT'$		$T' \rightarrow E$	$T' \rightarrow E$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Solution $\rightarrow$

$w = id + id * id \$$
 $\uparrow_a$



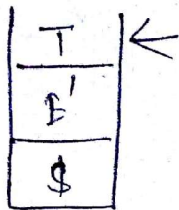
$w = id + id * id \$$
 $\uparrow$



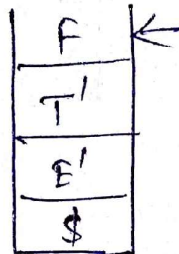
$$+ == +$$

∴ pop TOS and increment input pointer.

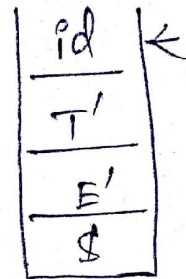
$$w = id + id * id \$$$



step (10)



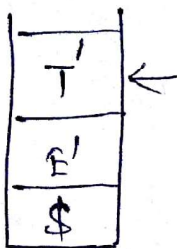
step (11)



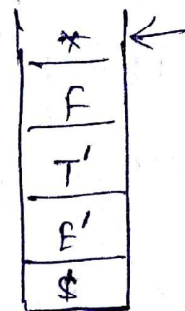
step (12)

$id == id$
∴ Pop TOS
and increment
input pointer

$$w = id + id * id \$$$



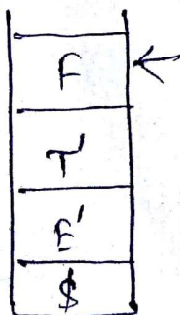
step (13)



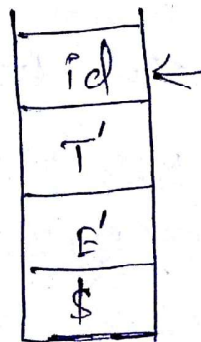
step (14)

$* == *$
∴ pop TOS and
increment input pointer

$$w = id + id * id \$$$



step (15)

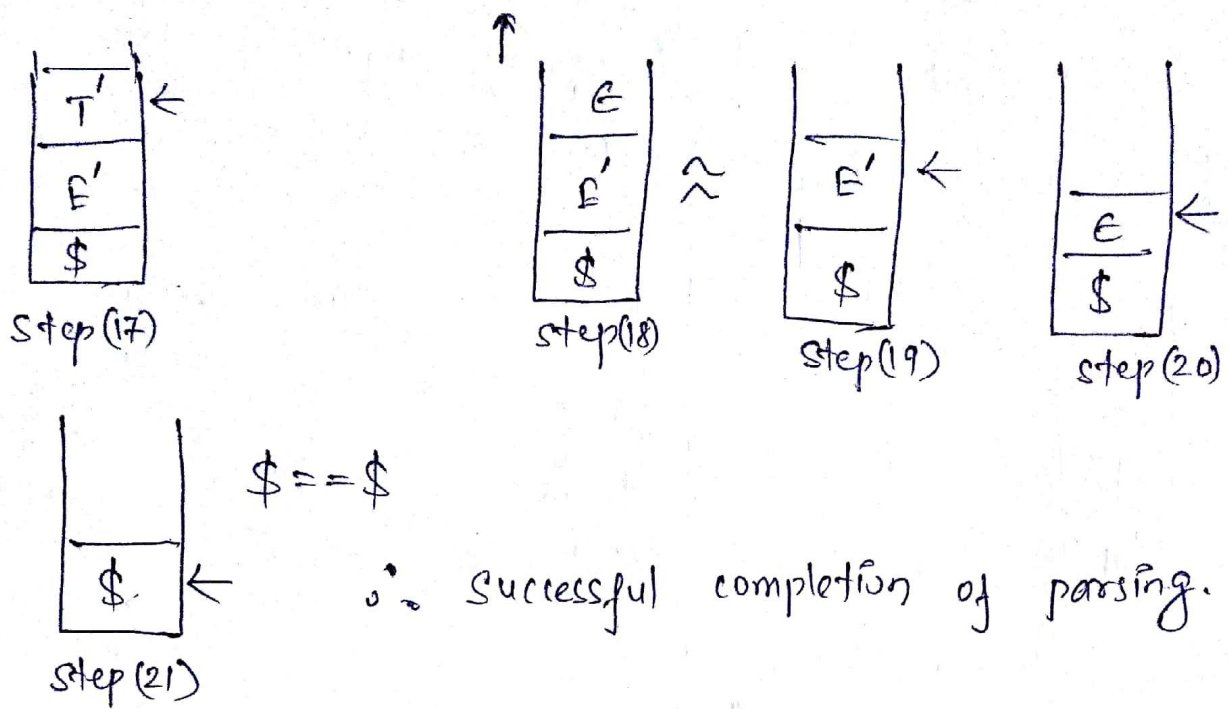


step (16)

$id == id$
∴ pop TOS and
increment input pointer

20.

$$w = id + id * id \$$$



∴ Successful completion of parsing.

Question 2

$$w = (id + id) * id$$

Input

$(id + id) * id \$$

Stack

$\$E$
 $\$E'T$
 $\$E'T'F$
 $\$E'T')E($

Production

$E \rightarrow TE'$
 $T \rightarrow FT'$
 $F \rightarrow (E)$

$id + id) * id \$$

$\$E'T')E$
 $\$E'T')E'T$
 $\$E'T')E'T'F$
 $\$E'T')E'T'id$

$E \rightarrow TE'$
 $T \rightarrow FT'$
 $F \rightarrow id$

$+ id) * id \$$

$\$E'T')E'T'$
 $\$E'T')E'$
 $\$E'T')E'T +$

$T' \rightarrow \epsilon$
 $E' \rightarrow +TE'$

id) * id \$
$$\begin{aligned} & \$ E' T') E' T' \\ & \$ E' T') E' T' F \\ & \$ E' T') E' T' id \end{aligned}$$

$$\begin{aligned} T & \rightarrow FT' \\ F & \rightarrow id \end{aligned}$$
) * id \$
$$\begin{aligned} & \$ E T') E' T' \\ & \$ E' T') \end{aligned}$$

$$\begin{aligned} T' & \rightarrow E \\ E' & \rightarrow E \end{aligned}$$
* id \$
$$\begin{aligned} & \$ E' T' \\ & \$ E' T' F + \end{aligned}$$
 $T' \rightarrow * FT'$ id \$
$$\begin{aligned} & \$ E' T' F \\ & \$ E' T' id \end{aligned}$$
 $F \rightarrow id$

\$

$$\begin{aligned} & \$ E' T' \\ & \$ \Rightarrow \text{Successful} \\ & \quad \text{completion.} \end{aligned}$$

$$\begin{aligned} T' & \rightarrow E \\ E' & \rightarrow E \end{aligned}$$
Question 3 $S \rightarrow (L)/a$ $L \rightarrow SL'$ $L' \rightarrow \epsilon / , SL'$

	a	,	(	)	\$
S	$S \rightarrow a$		$S \rightarrow (L)$		
L	$L \rightarrow SL'$		$L \rightarrow SL'$		
L'		$L' \rightarrow , SL'$		$L' \rightarrow \epsilon$	

 $w = (a, a, a)$

(22)

<u>Input</u>	<u>Stack</u>	<u>Production</u>
<u>(</u> a,a,a)\$	\$S	$S \rightarrow (L)$
a,a,a)\$	\$)L(	
a,a,a)\$	\$)L'	$L \rightarrow SL'$
	\$)L'S	$S \rightarrow a$
	\$)L'a	
,a,a)\$	\$)L'	$L' \rightarrow ,SL'$
	\$)L'S,	
a,a)\$	\$)L'S	$S \rightarrow a$
	\$)L'a	
,a)\$	\$)L'	$L' \rightarrow ,SL'$
	\$)L'S,	
a)\$	\$)L'S	$S \rightarrow a$
	\$)L'a	
)\$	\$)L'	$L' \rightarrow \epsilon$
	\$)	
\$	\$	successful completion.

Note $\Rightarrow$ Time complexity of LL(1) parsing algorithm is $O(n)$.

Because for every terminal we will take only one production.

LL(1) Parsing Table Construction



First()

Follow()

First() $\Rightarrow$ First(A) gives set of all terminals that may begin in strings derived by A.

Example 1: $S \rightarrow abc/def/ghi$

$$\text{First}(S) = \{a, d, g\}$$

Example 2: $S \rightarrow \epsilon$

$$\text{First}(S) = \{\epsilon\}$$

Example 3: $S \rightarrow ABC$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$C \rightarrow c$$

$$\text{First}(S) = \text{First}(ABC)$$

$$= \text{First}(A)$$

$$= \{a\}$$

$$\therefore \text{First}(A) = \{a\}$$

$$\text{First}(B) = \{b\}$$

$$\text{First}(C) = \{c\}$$

Example 4:

$$S \rightarrow ABC$$

$$A \rightarrow a/h$$

$$B \rightarrow b/d/e$$

$$C \rightarrow c/f/g$$

$$\text{First}(S) = \text{First}(ABC)$$

$$= \{a, h\}$$

$$\text{First}(A) = \{a, h\}$$

$$\text{First}(B) = \{b, d, e\}$$

$$\text{First}(C) = \{c, f, g\}$$

(24)

Example 5:

$$S \rightarrow ABC$$

$$A \rightarrow a/h/\epsilon$$

$$B \rightarrow b/d/e$$

$$C \rightarrow c/f/g$$

$$\text{First}(S) = \text{First}(ABC)$$

$$= \{a, h, b, d, e\}$$

$$\text{First}(A) = \{a, h, \epsilon\}$$

Example 6:

$$S \rightarrow ABC$$

$$A \rightarrow a/h/\epsilon$$

$$B \rightarrow b/d/e/\epsilon$$

$$C \rightarrow c/f/g$$

$$\text{First}(S) = \text{First}(ABC)$$

$$= \{a, h, b, d, e, c, f, g\}$$

$$\text{First}(B) = \{b, d, e, \epsilon\}$$

Example 7:

$$S \rightarrow ABC$$

$$A \rightarrow a/h/\epsilon$$

$$B \rightarrow b/d/e/\epsilon$$

$$C \rightarrow c/f/g/\epsilon$$

$$\text{First}(S) = \text{First}(ABC)$$

$$= \{a, h, b, d, e, c, f, g, \epsilon\}$$

$$\text{First}(C) = \{c, f, g, \epsilon\}$$

Rules to find out "First"

(1) If α is terminal, then $\text{first}(\alpha) = \alpha$

(2) If α is variable,

(i) $\alpha \rightarrow \epsilon$

$$\text{first}(\alpha) = \epsilon$$

(ii) $\alpha \rightarrow x_1 x_2 x_3$

$$x_1 \rightarrow a/b$$

$$x_2 \rightarrow c/d$$

$$x_3 \rightarrow e/f$$

$$\text{first}(\alpha) = \text{first}(x_1 x_2 x_3)$$

$$= \text{first}(x_1) - \epsilon \quad [\text{if } \text{first}(x_1) \text{ contain}]$$

$$= a, b$$

$$\cup$$

$$\text{first}(\alpha_2 \alpha_3)$$

$\Downarrow$

$$\text{first}(\alpha_2) - \epsilon \text{ [if first}(\alpha_2) \text{ contain]} \\ \cup \\ \text{first}(\alpha_3)$$

Question $\Rightarrow$ Find first for the following Grammar.

$$E \rightarrow TE'$$

$$E' \rightarrow \epsilon / + TE'$$

$$T \rightarrow FT'$$

$$T' \rightarrow \epsilon / * FT'$$

$$F \rightarrow \text{id} / (E)$$

Solution $\Rightarrow$

$$\text{first}(F) = \{\text{id}, (\}$$

$$\text{first}(T') = \{\epsilon, *\}$$

$$\begin{aligned} \text{first}(T) &= \text{first}(FT') \\ &= \text{first}(F) = \{\text{id}, (\} \end{aligned}$$

$$\text{first}(E') = \{\epsilon, +\}$$

$$\begin{aligned} \text{first}(E) &= \text{first}(TE') \\ &= \text{first}(T) = \{\text{id}, (\} \end{aligned}$$

Question $\Rightarrow$ Find first for the following Grammar.

$$S \rightarrow (L) / a$$

$$L \rightarrow SL'$$

$$L' \rightarrow \epsilon / , SL'$$

Solution:

$$\text{first}(L') = \{\epsilon, ,\}$$

$$\text{first}(L) = \{(, a\}$$

$$\text{first}(S) = \{(, a\}$$

Q6 Question $\Rightarrow$ Find First for the following Grammar.

$$S \rightarrow aBDh / EF / DBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC / \epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g / \epsilon$$

$$F \rightarrow f / \epsilon$$

Solution $\Rightarrow$ $\text{First}(F) = \{f, \epsilon\}$

$$\text{First}(E) = \{g, \epsilon\}$$

$$\text{First}(D) = \{g, f, \epsilon\}$$

$$\text{First}(C) = \{b, \epsilon\}$$

$$\text{First}(B) = \{c\}$$

$$\text{First}(S) = \{a, g, f, \epsilon, c\}$$

Question $\Rightarrow$ Find First for the following Grammar.

$$S \rightarrow AaAb / BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

Solution $\Rightarrow$

$$\text{First}(B) = \{\epsilon\}$$

$$\text{First}(A) = \{\epsilon\}$$

$$\text{First}(S) = \{a, b\}$$

Follow

Follow(A) gives set of all terminals that may follow immediately to the right of A.

Rules to find Follow

- (1) If A is start symbol,
Follow(A) = \$
- (2) If $S \rightarrow ABCD$, $B \rightarrow b$
Follow(A) = First(BCD).
= First(B)
= b
- (3) If $S \rightarrow BA$ (or) $S \rightarrow BAC$
 $C \rightarrow \epsilon$

then Follow(A) = Follow(S)

NOTE $\Rightarrow$ Follow never give epsilon ' ϵ '.

Question 1 $\Rightarrow$ Find First and Follow:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow \epsilon / +TE' \\ T &\rightarrow FT' \\ T' &\rightarrow \epsilon / *FT' \\ F &\rightarrow id / (E) \end{aligned}$$

Solution $\Rightarrow$

$$\begin{aligned} \text{First}(F) &= \{id, (\} \\ \text{First}(T') &= \{\epsilon, *\} \\ \text{First}(T) &= \{id, (\} \\ \text{First}(E') &= \{\epsilon, +\} \\ \text{First}(E) &= \{id, (\} \end{aligned}$$

$$\text{Follow}(E) = \{\$,)\}$$

$$\begin{aligned} \text{Follow}(E') &= \text{Follow}(E) \\ &= \{\$,)\} \end{aligned}$$

$$\begin{aligned} \text{Follow}(T) &= \text{First}(E') \\ &\Rightarrow \{+, \$,)\} \end{aligned}$$

ie +, Follow(E)

$$\text{Follow}(T') = \text{Follow}(T) \\ = \{+, \$,)\}$$

ie $+$, $\text{Follow}(E)$

$$\text{Follow}(F) = \text{First}(T') \\ = \{*, +, \$,)\}$$

ie $*$, $\text{Follow}(T)$

Question 2 $\Rightarrow$ Find First and Follow

$$S \rightarrow (L)/a$$

$$L \rightarrow SL'$$

$$L' \rightarrow \epsilon / , SL'$$

Solution $\Rightarrow$ $\text{First}(L') = \{\epsilon, ,\}$

$$\text{First}(L) = \{(, a\}$$

$$\text{First}(S) = \{(, a\}$$

$$\begin{aligned} \text{Follow}(S) &= \text{First}(L') \\ &= \{\epsilon, , \text{Follow}(L)\} \\ &= \{\epsilon, , \$\} \end{aligned}$$

$$\text{Follow}(L) = \{)\}$$

$$\begin{aligned} \text{Follow}(L') &= \text{Follow}(L) \\ &= \{)\} \end{aligned}$$

Question 3 $\Rightarrow$ Find First and Follow

$$S \rightarrow AaAb / BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

Solution $\Rightarrow$ $\text{First}(B) = \{\epsilon\}$

$$\text{First}(A) = \{\epsilon\}$$

$$\text{First}(S) = \{a, b\}$$

$$\text{Follow}(S) = \{\$\}$$

$$\text{Follow}(A) = \{a, b\}$$

$$\text{Follow}(B) = \{b, a\}$$

Question 4 $\Rightarrow$ Find first and follow

(27)

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC/\epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g/\epsilon$$

$$F \rightarrow f/\epsilon$$

Solution $\Rightarrow$ First (F) = {f, ϵ }

$$\text{First}(E) = \{g, \epsilon\}$$

$$\text{First}(D) = \{g, f, \epsilon\}$$

$$\text{First}(C) = \{b, \epsilon\}$$

$$\text{First}(B) = \{c\}$$

$$\text{First}(S) = \{a\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\begin{aligned} \text{Follow}(B) &= \text{First}(Dh) \\ &= \{g, f, h\} \end{aligned}$$

$$\begin{aligned} \text{Follow}(C) &= \text{Follow}(B) \\ &= \{g, f, h\} \end{aligned}$$

$$\text{Follow}(D) = \{h\}$$

$$\begin{aligned} \text{Follow}(E) &= \text{First}(F) \\ &= \{f, h\} \end{aligned}$$

$$\text{Follow}(F) = \text{Follow}(D) = \{h\}$$

Question 5 $\Rightarrow$ Find First and Follow

$$A \rightarrow BA'$$

$$A' \rightarrow *BA'/\epsilon$$

$$B \rightarrow CB'/\epsilon$$

$$B' \rightarrow *CB'/\epsilon$$

$$C \rightarrow +Ad/id$$

Solution $\Rightarrow$ First (C) = {+, id}

$$\text{First}(B') = \{*, \epsilon\}$$

$$\text{First}(B) = \{\epsilon, +, id\}$$

$$\text{First}(A') = \{*, \epsilon\}$$

$$\begin{aligned} \text{First}(A) &= \text{First}(BA') \\ &= \{+, id, *, \epsilon\} \end{aligned}$$

(30)

$$\text{Follow}(A) = \{\$, d\}$$

$$\begin{aligned}\text{Follow}(A') &= \text{Follow}(A) \\ &= \{\$, d\}\end{aligned}$$

$$\begin{aligned}\text{Follow}(B) &= \text{first}(A') \\ &= \{\#, \$, d\}\end{aligned}$$

$$\text{Follow}(B') = \text{Follow}(B) = \{\#, \$, d\}$$

$$\text{Follow}(C) = \text{first}(B') = \{\#, \$, d\}.$$

LL(1) Parsing Table Construction

For each production $A \rightarrow \alpha$, repeat following steps:

- (i) Add $A \rightarrow \alpha$ under $M[A, c]$, where $c \in \text{first}(\alpha)$
- (ii) Add $A \rightarrow \alpha$ under $M[A, d]$, where $d \in \text{Follow}(A)$ if $\text{first}(\alpha)$ contain ϵ .

Question $\Rightarrow$ Construct LL(1) parsing table for the following Grammar.

$$E \rightarrow TE'$$

$$E' \rightarrow \epsilon / + TE'$$

$$T \rightarrow FT'$$

$$T' \rightarrow \epsilon / * FT'$$

$$F \rightarrow id / (E).$$

Solution $\Rightarrow$

Non-terminal Symbol	Input Symbols					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow E$	$E' \rightarrow E$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow E$	$T' \rightarrow *FT'$		$T' \rightarrow E$	$T' \rightarrow E$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Note $\Rightarrow$ In the LL(1) parsing table, every entry contain maximum one production, then given grammar is LL(1) otherwise it is not LL(1).

Question 2 $\Rightarrow$ Construct LL(1) parsing table for the following Grammar.

$$S \rightarrow (L)/a$$

$$L \rightarrow SL'$$

$$L' \rightarrow \epsilon / , SL'$$

Solution $\Rightarrow$

	(	)	a	,	\$
S	$S \rightarrow (L)$		$S \rightarrow a$		
L	$L \rightarrow SL'$		$L \rightarrow SL'$		
L'		$L' \rightarrow \epsilon$		$L' \rightarrow , SL'$	

$\therefore$ Given grammar is LL(1).

Question 3 $\Rightarrow$ Check the following Grammar is LL(1) or not

$$S \rightarrow AaAb | BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

(32)

Solution $\Rightarrow$

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

$\therefore$ Given grammar is LL(1).

Question 4 $\Rightarrow$ Check the following grammar is LL(1) or not.

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC/\epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g/\epsilon$$

$$F \rightarrow f/\epsilon$$

Solution $\Rightarrow$

	a	b	c	g	f	h	\$
S	$S \rightarrow aBDh$						
B			$B \rightarrow cC$				
C		$C \rightarrow bC$		$C \rightarrow \epsilon$	$C \rightarrow \epsilon$	$C \rightarrow \epsilon$	
D				$D \rightarrow EF$	$D \rightarrow EF$	$D \rightarrow EF$	
E				$E \rightarrow g$	$E \rightarrow \epsilon$	$E \rightarrow \epsilon$	
F					$F \rightarrow f$	$F \rightarrow \epsilon$	

$\therefore$ Given grammar is LL(1).

Question 5 $\Rightarrow$ Check the following Grammar is LL(1) or not.

$$S \rightarrow A$$

$$A \rightarrow aBA'$$

$$A' \rightarrow dA' / \epsilon$$

$$B \rightarrow b$$

$$C \rightarrow g$$

Solution $\Rightarrow$

	a	b	d	g	\$
S	$S \rightarrow A$				
A	$A \rightarrow aBA'$				
A'			$A' \rightarrow dA'$		$A' \rightarrow \epsilon$
B		$B \rightarrow b$			
C				$C \rightarrow g$	

$\therefore$ Given grammar is LL(1).

Question 6 $\Rightarrow$ Check the following Grammar is LL(1) or not.

$$S \rightarrow aSbS / bSaS / \epsilon$$

Solution $\Rightarrow$

	a	b	\$
S	$S \rightarrow \epsilon$ $S \rightarrow aSbS$	$S \rightarrow \epsilon$ $S \rightarrow bSaS$	$S \rightarrow \epsilon$

$\therefore$ It is not LL(1)

Because single cell contain multiple entries.

Predictive LL(1) Parser

LL(1)

- * First L means the input is scanned from left to right.
- * Second L means it uses leftmost derivation for input string.
- * Number 1 means it uses only one input symbol (lookahead) to predict the parsing process.

Q For the following grammar, find FIRST and FOLLOW sets for each of non-terminal.

$$S \rightarrow aAB|bA|\epsilon$$

$$A \rightarrow aAb|\epsilon$$

$$B \rightarrow bB|\epsilon$$

Soln $\text{first}(B) = \{b, \epsilon\}$

$$\text{first}(A) = \{a, \epsilon\}$$

$$\text{first}(S) = \{a, b, \epsilon\}$$

$$\text{follow}(S) = \{\$ \}$$

$$\begin{aligned}\text{follow}(A) &= \{b, \text{first}(B), \text{follow}(S)\} \\ &= \{b, \$ \}\end{aligned}$$

$$\begin{aligned}\text{follow}(B) &= \text{follow}(S) \\ &= \{\$ \}\end{aligned}$$

Q Consider the grammar $\Rightarrow$

$$S \rightarrow ACB \mid CbB \mid Ba$$

$$A \rightarrow da \mid BC$$

$$B \rightarrow g \mid \epsilon$$

$$C \rightarrow h \mid \epsilon$$

Calculate FIRST and FOLLOW.

Soln $\text{First}(C) = \{h, \epsilon\}$

$$\text{First}(B) = \{g, \epsilon\}$$

$$\begin{aligned}\text{First}(A) &= \{d, \text{First}(B)\} \\ &= \{d, g, \text{First}(C)\} \\ &= \{d, g, h, \epsilon\}\end{aligned}$$

$$\begin{aligned}\text{First}(S) &= \{\text{First}(A), \text{First}(C), \text{First}(B)\} \\ &= \{d, g, h, \epsilon, b, a\}\end{aligned}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\begin{aligned}\text{Follow}(A) &= \text{First}(C) = \{h, \text{First}(B)\} \\ &= \{h, g, \text{Follow}(S)\} \\ &= \{h, g, \$ \}\end{aligned}$$

$$\begin{aligned}\text{Follow}(B) &= \{a, \text{Follow}(S), \text{First}(C)\} \\ &= \{a, \$, h, \text{Follow}(A)\} \\ &= \{a, \$, h, g\}\end{aligned}$$

$$\begin{aligned}\text{Follow}(C) &= \{b, \text{Follow}(A), \text{First}(B)\} \\ &= \{b, h, g, \$, \epsilon\}\end{aligned}$$

Q Check whether the given grammar is LL(1)?
Remove left recursion and then again verify whether it is LL(1)?

$$S \rightarrow Aa|b$$

$$A \rightarrow Ac|Sd|E$$

Soln $A \rightarrow Ac$ is left recursive. $\therefore$ not LL(1).

Left recursion can be removed as follows:

If $A \rightarrow A\alpha/B$, then $A \rightarrow \beta A'$
 $A' \rightarrow \alpha A'/E$

$$\therefore A \rightarrow Ac|Sd|E$$

$$\downarrow \quad \downarrow \downarrow \quad \downarrow \downarrow \quad \downarrow$$

$$A \quad A\alpha \quad \beta \quad \beta$$

$$\therefore A \rightarrow SdA'/EA'$$

$$A' \rightarrow cA'/E$$

$\therefore$ production becomes

$$S \rightarrow Aa|b$$

$$A \rightarrow SdA'/EA'$$

$$A' \rightarrow cA'/E$$

~~First~~ Now to check for LL(1).

Compare with $A \rightarrow \alpha$

$$\therefore S \rightarrow Aa$$

$$\downarrow \quad \downarrow$$

$$A \quad \alpha$$

$$\text{First}(Aa) = \{b, a, c\}$$

$$\begin{array}{c} S \rightarrow b \\ \downarrow \\ A \end{array} \quad \begin{array}{c} \downarrow \\ \alpha \end{array}$$

$$\text{First}(b) = \{b\}$$

$$\begin{array}{c} A \rightarrow SdA' \\ \downarrow \quad \downarrow \\ A \quad \alpha \end{array}$$

$$\text{First}(\alpha) \text{ i.e. } \text{First}(SdA') = \{b, \text{First}(A)\}$$

(OR)

Method to check for LL(1).

$$\text{First}(A') = \{c, e\}$$

$$\begin{aligned} \text{First}(A) &= \{\text{First}(S), \text{First}(A')\} \\ &= \{a, b, c, e\} \end{aligned}$$

$$\text{First}(S) = \{a, b, c\}$$

$$\text{Follow}(S) = \{\$, d\}$$

$$\text{Follow}(A) = \{a\}$$

$$\text{Follow}(A') = \text{Follow}(A) = \{a\}$$

$\therefore$

	a	b	c	d	\$
S	$S \rightarrow Aa$	$S \rightarrow b$			
A	$A \rightarrow SdA'$ $A \rightarrow eA'$	$A \rightarrow SdA'$			
A'	$A' \rightarrow cA'$ $A' \rightarrow e$				

Table contain multiple entries.

$\therefore$ The Given grammar is not LL(1).

Ques → Design LL(1) parsing table for the following grammar.

$$A \rightarrow AcB / cC / C$$

$$B \rightarrow bB / \epsilon d$$

$$C \rightarrow CaB / BbB / B$$

Soln

$A \rightarrow AcB / cC / C$ is left recursive. We will eliminate left recursion.

$$A \rightarrow AcB / cC / C$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$A \quad A \quad \alpha \quad \beta \quad \beta$$

$$\therefore \text{If } A \rightarrow A\alpha / \beta, \text{ then } A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \epsilon$$

$\therefore A \rightarrow AcB / cC / C$ becomes

$$A \rightarrow cCA' / CA'$$

$$A' \rightarrow cBA' / \epsilon$$

The rule now becomes: →

$$A \rightarrow cCA' / CA'$$

$$A' \rightarrow cBA' / \epsilon$$

$$B \rightarrow bB / \epsilon d$$

and $C \rightarrow CaB / BbB / B$ is left recursive.

$$C \rightarrow CaB / BbB / B$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$A \quad A \quad \alpha \quad \beta \quad \beta$$

$$\therefore C \rightarrow BbBc' / Bc'$$

$$c' \rightarrow aBc' / \epsilon$$

Finally the rule becomes

$$A \rightarrow cCA' / CA'$$

$$A' \rightarrow cBA' / \epsilon$$

$$B \rightarrow bB / id$$

$$C \rightarrow BbBc' / Bc'$$

$$c' \rightarrow aBc' / \epsilon$$

Now consider the rule $C \rightarrow BbBc' / Bc'$ has left factoring.

$$\begin{array}{ccccccc} C & \rightarrow & B & b & B & c' & / Bc' \\ \downarrow & & \downarrow & & \downarrow & & \downarrow \\ \alpha & & \alpha & & \beta_1 & & \beta_2 \end{array}$$

$$\text{If } A \rightarrow \alpha B_1 / \alpha B_2 / \dots, \text{ then } A \rightarrow \alpha A' \\ A' \rightarrow B_1 / B_2 / \dots$$

$$\therefore C \rightarrow Bc''$$

$$c'' \rightarrow bBc' / c'$$

$\therefore$ The grammar can be completely written as $\Rightarrow$

$$A \rightarrow cCA' / CA'$$

$$A' \rightarrow cBA' / \epsilon$$

$$B \rightarrow bB / id$$

$$C \rightarrow Bc''$$

$$c'' \rightarrow bBc' / c'$$

$$c' \rightarrow aBc' / \epsilon$$

$$\text{First}(c') = \{a, \epsilon\}$$

$$\text{First}(c'') = \{b, \text{First}(c')\}$$

$$= \{b, a, \epsilon\}$$

$$\text{First}(c) = \text{First}(B) = \{b, id\}$$

$$\text{First}(B) = \{b, id\}$$

$$\text{First}(A') = \{c, \epsilon\}$$

$$\text{First}(A) = \{c, \text{First}(c')\}$$

$$= \{c, b, id\}$$

$$\text{Follow}(A) = \{\$, \}$$

$$\text{Follow}(A') = \text{Follow}(A) = \{\$, \}$$

$$\text{Follow}(B) = \text{First}(A'), \text{First}(c''),$$

$$\text{First}(c')$$

$$= \{c, \$, b, a, id\}$$

$$\text{Follow}(c) = \{\text{First}(A')\}$$

$$= \{c, \text{Follow}(A)\}$$

$$= \{c, \$\}$$

$$\text{Follow}(c'') = \text{Follow}(c) = \{c, \$\}$$

$$\text{Follow}(c') = \text{Follow}(c'') = \{c, \$\}$$

$\therefore$ Predictive Parsing table is:

	a	b	c	id	\$
A		$A \rightarrow CA'$	$A \rightarrow cCA'$	$A \rightarrow CA'$	
A'			$A' \rightarrow cBA'$		$A' \rightarrow \epsilon$
B		$B \rightarrow bB$		$B \rightarrow id$	
C		$C \rightarrow Bc''$		$C \rightarrow Bc''$	
c'	$c' \rightarrow aBc'$		$c' \rightarrow \epsilon$	ϵ	$c' \rightarrow \epsilon$
c''	$c'' \rightarrow c'$	$c'' \rightarrow bBc'$	$c'' \rightarrow c'$		$c'' \rightarrow c'$

Q Consider the grammar

$$S \rightarrow iCtSA/a$$

$$A \rightarrow eS/\epsilon$$

$$C \rightarrow b$$

whether it is LL(1) grammar? Give the explanation whether (i, t, e, b, a) are terminal symbols.

Sol

$$S \rightarrow iCtSA/a$$

$$A \rightarrow eS/\epsilon$$

$$C \rightarrow b$$

$$\text{First}(C) = \{b\}$$

$$\text{First}(A) = \{e, \epsilon\}$$

$$\text{First}(S) = \{i, a\}$$

$$\text{Follow}(S) = \$, \text{First}(A), \text{Follow}(A) \\ = \{\$, e\}$$

$$\text{Follow}(A) = \text{Follow}(S) = \{\$, e\}$$

$$\text{Follow}(C) = \{t\}$$

The predictive parsing table is

	a	b	e	t	i	\$
S	$S \rightarrow a$				$S \rightarrow iCtSA$	
A			$A \rightarrow eS$ $A \rightarrow \epsilon$			$A \rightarrow \epsilon$
C		$C \rightarrow b$				

As we have got multiple entries in $M[A, e]$.

∴ Grammar is not LL(1) grammar.

The (i, t, e, b, a) are the terminal symbols because they do not derive any production rule.

Q Construct parsing table for the following grammar:

$$S \rightarrow aXYb$$

$$X \rightarrow c|e$$

$$Y \rightarrow d|e$$

Solution

$$\text{First}(Y) = \{d, e\}$$

$$\text{First}(X) = \{c, e\}$$

$$\text{First}(S) = \{a\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\begin{aligned}\text{Follow}(X) &= \text{First}(Yb) \\ &= \{d, b\}\end{aligned}$$

$$\text{Follow}(Y) = \{b\}$$

The parsing table is

	a	b	c	d	\$
S	$S \rightarrow aXYb$				
X		$X \rightarrow e$	$X \rightarrow c$	$X \rightarrow e$	
Y		$Y \rightarrow e$		$Y \rightarrow d$	

Q. Consider the grammar

$$\text{temp} \rightarrow \text{atom} | \text{list}$$

$$\text{atom} \rightarrow \text{number} | \text{identifier}$$

$$\text{list} \rightarrow (\text{temp-seg})$$

$$\text{temp-seg} \rightarrow \text{temp}, \text{temp-seg} | \text{temp}$$

(i) Left factor this grammar

(ii) Construct FIRST and FOLLOW sets for the non-terminals

(iii) Show that resultant grammar is LL(1).

(iv) Construct LL(1) parsing table for the resultant grammar.

Soln For the given grammar,

Set of terminals $T = \{\text{number}, \text{identifier}, (,), ,\}$

Set of nonterminals $V = \{\text{textp}, \text{atom}, \text{list}, \text{textp-seg}\}$

Grammar is $\text{textp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{number} \mid \text{identifier}$

$\text{list} \rightarrow (\text{textp-seg})$

$\text{textp-seg} \rightarrow \text{textp}, \text{textp-seg} \mid \text{textp}$

(i) Consider the production rule

$$\begin{array}{ccccccc} \text{textp-seg} & \rightarrow & \text{textp}, & \text{textp-seg} & \mid & \text{textp} & \\ \downarrow & & \downarrow & \downarrow & & \downarrow & \downarrow \\ A & & \alpha & \beta_1 & & \alpha & \beta \text{ (ie } \epsilon) \end{array}$$

If $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots$, then $A \rightarrow \alpha A'$
 $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots$

$\therefore \text{textp-seg} \rightarrow \text{textp textp-seg}'$

$\text{textp-seg}' \rightarrow , \text{textp-seg} \mid \epsilon$

The production after removal of Left Factoring is

$\text{textp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{number} \mid \text{identifier}$

$$\text{list} \rightarrow (\text{temp-seg})$$

$$\text{temp-seg} \rightarrow \text{temp temp-seg}'$$

$$\text{temp-seg}' \rightarrow , \text{temp-seg} / \epsilon$$

Now calculate FIRST and FOLLOW for each non-terminals.

$$\text{First}(\text{temp}) = \{\text{number}, \text{identifier}, (\}$$

$$\text{First}(\text{atom}) = \{\text{number}, \text{identifier}\}$$

$$\text{First}(\text{list}) = \{ (\}$$

$$\begin{aligned} \text{First}(\text{temp-seg}) &= \{\text{First}(\text{temp})\} \\ &= \{\text{number}, \text{identifier}, (\} \end{aligned}$$

$$\text{First}(\text{temp-seg}') = \{ , , \epsilon \}$$

$$\begin{aligned} \text{Follow}(\text{temp}) &= \{ \$, \text{First}(\text{temp-seg}') \} \\ &= \{ \$, , , \text{Follow}(\text{temp-seg}) \} \\ &= \{ \$, , ,) \} \end{aligned}$$

$$\begin{aligned} \text{Follow}(\text{atom}) &= \text{Follow}(\text{temp}) \\ &= \{ \$, , ,) \} \end{aligned}$$

$$\begin{aligned} \text{Follow}(\text{list}) &= \text{Follow}(\text{temp}) \\ &= \{ \$, , ,) \} \end{aligned}$$

$$\begin{aligned} \text{Follow}(\text{temp-seg}) &= \{ , , \text{Follow}(\text{temp-seg}') \} \\ &= \{ , ,) \} \end{aligned}$$

$$\text{Follow}(\text{temp-seg}') = \text{Follow}(\text{temp-seg}) = \{)\}$$

Now check for LL(1).

$$\begin{array}{ccc} \text{temp} & \longrightarrow & \text{atom} \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(\text{atom}) = \{\text{number}, \text{identifier}\}$$

$$\begin{array}{ccc} \text{temp} & \longrightarrow & \text{list} \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(\text{list}) = \{(\}$$

$$\begin{array}{ccc} \text{atom} & \longrightarrow & \text{number} \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(\text{number}) = \{\text{number}\}$$

$$\begin{array}{ccc} \text{atom} & \longrightarrow & \text{identifier} \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(\text{identifier}) = \{\text{identifier}\}$$

$$\begin{array}{ccc} \text{list} & \longrightarrow & (\text{temp-seg}) \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}((\text{temp-seg})) = \{(\}$$

$$\begin{array}{ccc} \text{temp-seg} & \longrightarrow & \text{temp temp-seg}' \\ \downarrow & & \downarrow \\ A & & \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(\text{temp temp-seg}')$$

$$= \text{First}(\text{temp})$$

$$= \{ \text{number, identifier, (} \}$$

$$\begin{array}{c} \text{temp-seg}' \rightarrow , \text{temp-seg} \\ \downarrow \quad \quad \downarrow \\ A \quad \quad \alpha \end{array}$$

$$\text{First}(\alpha) = \text{First}(, \text{temp-seg}) = \{ , \}$$

$$\begin{array}{c} \text{temp-seg}' \rightarrow \epsilon \\ \downarrow \quad \quad \downarrow \\ A \quad \quad \alpha \end{array}$$

$$\text{Follow}(\text{temp-seg}') = \{) \}$$

$\therefore$ The predictive parsing table is

	number	identifier	,	(	)	\$
temp	temp $\rightarrow$ atom	temp $\rightarrow$ atom		temp $\rightarrow$ list		
atom	atom $\rightarrow$ number	atom $\rightarrow$ identifier				
list				list $\rightarrow$ (temp-seg)		
temp-seg	temp-seg $\rightarrow$ temp temp-seg	temp-seg $\rightarrow$ temp temp-seg'		temp-seg $\rightarrow$ temp temp-seg'		
temp-seg'			temp-seg' $\rightarrow$, temp-seg		temp-seg' $\rightarrow$ ϵ	

As each cell in the above table contains unique entry,

$\therefore$ the given grammar is LL(1).

Q Consider the following grammar :

$$S' = S\#$$

$$S \rightarrow ABC$$

$$A \rightarrow a/bbD$$

$$B \rightarrow a/\epsilon$$

$$C \rightarrow b/\epsilon$$

$$D \rightarrow c/\epsilon$$

Construct the first and follow sets for the grammar also design LL(1) parsing table for the grammar.

Solution:->

$$\text{First}(D) = \{c, \epsilon\}$$

$$\text{First}(C) = \{b, \epsilon\}$$

$$\text{First}(B) = \{a, \epsilon\}$$

$$\text{First}(A) = \{a, b\}$$

$$\begin{aligned} \text{First}(S) &= \text{First}(ABC) \\ &= \{a, b\} \end{aligned}$$

$$\begin{aligned} \text{First}(S') &= \text{First}(S\#) \\ &= \{a, b\} \end{aligned}$$

$$\text{Follow}(S') = \{\$ \}$$

$$\text{Follow}(S) = \{\# \}$$

$$\begin{aligned} \text{Follow}(A) &= \text{First}(BC) \\ &= \{a, b, \# \} \end{aligned}$$

$$\begin{aligned} \text{Follow}(B) &= \text{First}(C) \\ &= \{b, \# \} \end{aligned}$$

$$\text{Follow}(C) = \text{Follow}(S) = \{\# \}$$

$$\begin{aligned} \text{Follow}(D) &= \text{Follow}(A) \\ &= \{a, b, \# \} \end{aligned}$$

	a	b	c	#	\$
S'	$S' \Rightarrow S\#$	$S' \Rightarrow S\#$			
S	$S \rightarrow ABC$	$S \rightarrow ABC$			
A	$A \rightarrow a$	$A \rightarrow bbD$			
B	$B \rightarrow a$	$B \rightarrow \epsilon$		$B \rightarrow \epsilon$	
C		$C \rightarrow b$		$C \rightarrow \epsilon$	
D	$D \rightarrow \epsilon$	$D \rightarrow \epsilon$	$D \rightarrow c$	$D \rightarrow \epsilon$	